

lab13(SSU)

October 15, 2025

1 Lab Assignment 13: Examining the Therapeutic Touch

This assignment involves using statistical modeling, hypothesis testing, and simulation to analyze data and draw conclusions

Recommended Reading:

- [Empirical Distribution of a Statistic](#)
- [Random Sampling in Python](#)
- [Python Reference](#)

After such an extensive introduction to programming for data science, we are finally moving into the section of the course where we can apply our new skills to answer real questions.

In this lab, we'll use testing techniques that were introduced in lecture to test the idea of the therapeutic touch, the idea that some practitioner can feel and massage your human energy field.

```
[74]: !apt-get install texlive texlive-xetex texlive-latex-extra pandoc
      !pip install pypandoc
      !pip install datascience
```

```
shell-init: error retrieving current directory: getcwd: cannot access parent
directories: Transport endpoint is not connected
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
texlive is already the newest version (2021.20220204-1).
texlive-latex-extra is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 38 not upgraded.
shell-init: error retrieving current directory: getcwd: cannot access parent
directories: Transport endpoint is not connected
shell-init: error retrieving current directory: getcwd: cannot access parent
directories: Transport endpoint is not connected
Traceback (most recent call last):
  File "/usr/local/bin/pip3", line 4, in <module>
    from pip._internal.cli.main import main
  File "/usr/local/lib/python3.12/dist-packages/pip/_internal/cli/main.py", line
11, in <module>
```

```

    from pip._internal.cli.autocompletion import autocomplete
File "/usr/local/lib/python3.12/dist-
packages/pip/_internal/cli/autocompletion.py", line 10, in <module>
    from pip._internal.cli.main_parser import create_main_parser
File "/usr/local/lib/python3.12/dist-
packages/pip/_internal/cli/main_parser.py", line 9, in <module>
    from pip._internal.build_env import get_runnable_pip
File "/usr/local/lib/python3.12/dist-packages/pip/_internal/build_env.py",
line 19, in <module>
    from pip._internal.cli.spinners import open_spinner
File "/usr/local/lib/python3.12/dist-packages/pip/_internal/cli/spinners.py",
line 9, in <module>
    from pip._internal.utils.logging import get_indentation
File "/usr/local/lib/python3.12/dist-packages/pip/_internal/utils/logging.py",
line 13, in <module>
    from pip._vendor.rich.console import (
File "/usr/local/lib/python3.12/dist-packages/pip/_vendor/rich/__init__.py",
line 17, in <module>
    _IMPORT_CWD = os.path.abspath(os.getcwd())
                                ~~~~~~

```

```

OSError: [Errno 107] Transport endpoint is not connected
shell-init: error retrieving current directory: getcwd: cannot access parent
directories: Transport endpoint is not connected
shell-init: error retrieving current directory: getcwd: cannot access parent
directories: Transport endpoint is not connected

```

Traceback (most recent call last):

```

File "/usr/local/bin/pip3", line 4, in <module>
    from pip._internal.cli.main import main
File "/usr/local/lib/python3.12/dist-packages/pip/_internal/cli/main.py", line
11, in <module>
    from pip._internal.cli.autocompletion import autocomplete
File "/usr/local/lib/python3.12/dist-
packages/pip/_internal/cli/autocompletion.py", line 10, in <module>
    from pip._internal.cli.main_parser import create_main_parser
File "/usr/local/lib/python3.12/dist-
packages/pip/_internal/cli/main_parser.py", line 9, in <module>
    from pip._internal.build_env import get_runnable_pip
File "/usr/local/lib/python3.12/dist-packages/pip/_internal/build_env.py",
line 19, in <module>
    from pip._internal.cli.spinners import open_spinner
File "/usr/local/lib/python3.12/dist-packages/pip/_internal/cli/spinners.py",
line 9, in <module>
    from pip._internal.utils.logging import get_indentation
File "/usr/local/lib/python3.12/dist-packages/pip/_internal/utils/logging.py",
line 13, in <module>
    from pip._vendor.rich.console import (
File "/usr/local/lib/python3.12/dist-packages/pip/_vendor/rich/__init__.py",
line 17, in <module>

```

```
_IMPORT_CWD = os.path.abspath(os.getcwd())
                        ~~~~~~
```

OSError: [Errno 107] Transport endpoint is not connected

```
[75]: # Connect Google Drive to Colab so you can access your files
from google.colab import drive
drive.mount('/content/drive')

import os
os.chdir('/content/drive/MyDrive/Colab Notebooks/')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[76]: # Run this cell, but please don't change it.

# These lines import the Numpy and Datascience modules.
import numpy as np
from datascience import *

# These lines do some fancy plotting magic
import matplotlib
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
import warnings
warnings.simplefilter('ignore', FutureWarning)
from matplotlib import patches
from ipywidgets import interact, interactive, fixed
import ipywidgets as widgets
```

1.1 1. What is the Therapeutic Touch

The Therapeutic Touch (TT) is the idea that everyone can feel the Human Energy Field (HEF) around individuals. Those who practice TT have described different people's HEFs as "warm as Jell-O" and "tactile as taffy."

TT was a popular technique used throughout the 20th century that was touted as a great way to bring balance to a person's health. Certain practitioners claim they have the ability to feel the HEF and can massage it in order to promote health and relaxation in individuals.

1.1.1 Emily Rosa

[Emily Rosa](#) was a 4th grade student who was very familiar with the world of TT, thanks to her parents, who were both medical practitioners and skeptics of TT.

For her 4th grade science fair project, Emily decided to test whether or not TT practitioners could truly interact with a person's HEF. She later went on to publish her work in TT, becoming the youngest person to have a research paper published in a peer reviewed medical journal.

1.1.2 Emily's Experiment

Emily's experiment was clean, simple, and effective. Due to her parents' occupations in the medical field, she had wide access to people who claimed to be TT practitioners.

Emily took 21 TT practitioners and used them for her science experiment. She would take a TT practitioner and ask them to extend their hands through a screen (which they can't see through). Emily would be on the other side and would flip a fair coin. Depending on how the coin landed, she would put out either her left hand or her right hand. The TT practitioner would then have to answer which hand Emily put out. If a practitioner could truly interact with a person's HEF, it would be expected that they answered correctly.

Overall, through 210 samples, the practitioner picked the correct hand 44% of the time.

Emily's main goal here was to test whether or not the TT practitioners' guesses were random, like the flip of a coin. In most medical experiments, this is the norm. **We want to test whether or not the treatment has an effect, *not* whether or not the treatment actually works.**

We will now begin to formulate this experiment in terms of the terminology we learned in this course.

Question 1.1: Describe Emily's [model](#) for how likely the TT practitioners are to choose the correct hand. What alternative model is her model meant to discredit?

If you are able, check in with fellow peers, the discussion forum, or your lab TA/AIs, to come to a conclusion.

Emily's model is that TT practitioners simply flip a coin, with a 50% chance of getting it right.

The counter-model she seeks to refute is that they possess a special ability to sense the human energy field, giving them a better chance of getting it right than simply flipping a coin.

Question 1.2: Remember that the practitioner got the correct answer 44% (0.44) of the time. According to Emily's model, on average, what proportion of times do we expect the practitioner to guess the correct hand? Make sure your answer is a number between 0 and 1.

```
[77]: expected_proportion_correct = 0.5
      expected_proportion_correct
```

```
[77]: 0.5
```

The goal now is to see if our deviation from this expected proportion of correct answers is due to something other than chance.

Question 1.3: We usually use a statistic to help determine which model the evidence points towards. What is a statistic that we can use to compare outcomes under Emily's model to what was observed? Assign `valid_stat` to an array of integer(s) representing test statistics that Emily can use:

1. The difference between the expected percent correct and the actual percent correct
2. The absolute difference between the expected percent correct and the actual percent correct
3. The sum of the expected percent correct and the actual percent correct

NOTE: Make sure to use `make_array` to create your array of integer(s)!

Hint: What should the domain (possible x values) be for the distribution of our test statistics?

```
[78]: valid_stat = make_array(2)
      valid_stat
```

```
[78]: array([2])
```

Question 1.4: Why is the statistic from Question 1.3 the appropriate choice for comparing outcomes in Emily’s experiment? How does it relate to the models you defined in Question 1.1?

We use the absolute difference between the expected rate (50%) and the actual rate (44%) as our statistic. This method is appropriate because a large difference suggests that TT practitioners perform better than simply guessing (the alternative model), while a small difference supports Emily’s model, which suggests that guessing is the norm. Using the absolute difference measures the magnitude of the difference from 50%, regardless of whether the actual rate is higher or lower.

Question 1.5: Define the function `statistic` which takes in an expected proportion and an actual proportion, and returns the value of the statistic chosen in Question 1.3. Assume that the argument takes in proportions, but return your answer as a percentage.

Hint: Remember we are asking for a **percentage**, not a proportion.

```
[79]: def statistic(expected_prop, actual_prop):
      return abs(expected_prop - actual_prop) * 100
```

Question 1.6: Use your newly defined function to calculate the observed statistic from Emily’s experiment.

```
[80]: observed_statistic = statistic(0.5, 0.44)
      observed_statistic
```

```
[80]: 6.0
```

Is this observed statistic consistent with what we expect to see under Emily’s model?

In order to answer this question, we must simulate the experiment as though Emily’s model was correct, and calculate our statistic for every simulation.

1.1.3 sample_proportions

`sample_proportions` can be used to randomly sample from multiple categories when you know the proportion of data points that are expected to fall in each category. `sample_proportions` takes two arguments: the sample size and an array of proportions corresponding to each category in the population (should sum to 1).

Consider flipping a fair coin, where the two outcomes (coin lands heads and coin lands tails) occur with an equal chance. We expect that half of all coin flips will land heads, and half of all coin flips will land tails.

Run the following cell to see the simulation of 10 flips of a fair coin. Let the first item of `coin_proportions` be the proportion of heads and the second item of `coin_proportions` be the

proportion of tails.

Observe what happens when you run this cell multiple times—the proportion of coin flips that land heads and tails appears to change, as you are simulating flipping 10 coins each time!

```
[81]: coin_proportions = make_array(0.5, 0.5)
      ten_flips = sample_proportions(10, coin_proportions)
      ten_flips
```

```
[81]: array([ 0.7,  0.3])
```

`sample_proportions` returns an array that is the same length as the proportion array that is passed through. It contains the proportion of each category that appears in the sample.

In our example, the first item of `ten_flips` is the simulated proportion of heads and the second item of `ten_flips` is the simulated proportion of tails.

```
[82]: simulated_proportion_heads = ten_flips.item(0)
      simulated_proportion_tails = ten_flips.item(1)

      print("In our simulation, " + str(simulated_proportion_heads) + " of flips were_
      ↪heads and " \
            + str(simulated_proportion_tails) + " of flips were tails.")
```

In our simulation, 0.7 of flips were heads and 0.3 of flips were tails.

Question 1.7: To begin simulating, we should start by creating a representation of Emily's model to use for our simulation. This will be an array with two items in it. The first item should be the proportion of times a TT practitioner picks the correct hand, assuming that Emily's model was correct. The second item should be the proportion of times, under the same assumption, that the TT practitioner picks the incorrect hand. Assign `model_proportions` to this array.

After this, we can simulate 210 hand choices, as Emily evaluated in real life, and find a single statistic to summarize this instance of the simulation. Use the `sample_proportions` function and assign the **proportion of correct hand choices** (out of 210) to `simulation_proportion_correct`. Lastly, use your `statistic` function to assign `one_statistic` to the value of the statistic for this one simulation.

Hint: `sample_proportions` usage can be found on the [Python Reference](#).

```
[83]: # This saves the random state of our code so that we can
      # generate the same numbers each time we run the code.
      # Please do not change this line.
      np.random.seed(16)

      model_proportions = make_array(0.5, 0.5)
      simulation_proportion_correct = sample_proportions(210, model_proportions).
      ↪item(0)
      one_statistic = statistic(0.5, simulation_proportion_correct)
      one_statistic
```

[83]: 4.285714285714287

Question 1.8: Let's now see what the distribution of statistics is actually like under Emily's model.

Define the function `simulation_and_statistic` to take in the `model_proportions` array and the expected proportion of times a TT practitioner would guess a hand correctly under Emily's model. The function should simulate Emily running through the experiment 210 times and return the statistic of this one simulation.

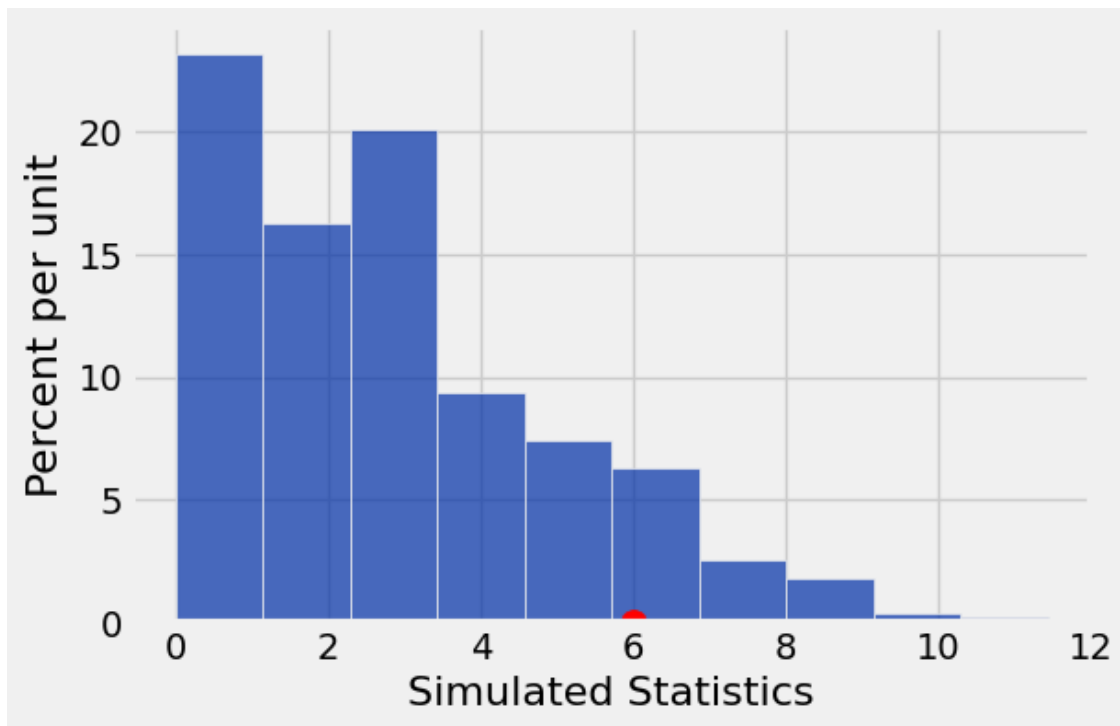
Hint: This should follow the same pattern as the code you did in the previous problem.

```
[84]: def simulation_and_statistic(model_proportions, expected_proportion_correct):  
      '''Simulates 210 TT hand choices under Emily's model.  
      Returns one statistic from the simulation.'''  
      return statistic(expected_proportion_correct, sample_proportions(210,  
      ↪model_proportions).item(0))
```

```
[85]: num_repetitions = 1000  
  
      simulated_statistics = make_array()  
  
      for i in np.arange(num_repetitions):  
          simulated_statistics = np.append(simulated_statistics,  
          ↪simulation_and_statistic(model_proportions, 0.5))
```

Let's view the distribution of the simulated statistics under Emily's model, and visually compare where the observed statistic lies relative to the simulated statistics.

```
[86]: t = Table().with_column('Simulated Statistics', simulated_statistics)  
      t.hist()  
      plt.scatter(observed_statistic, 0, color='red', s=100, zorder=2);
```



We can make a visual argument as to whether we believe the observed statistic is consistent with Emily's model. Here, since larger values of the test statistic suggest the alternative model (where the chance of guessing the correct hand is something other than 50%), we can formalize our analysis by finding what proportion of simulated statistics were as large or larger than our observed test statistic (the area at or to the right of the observed test statistic). If this area is small enough, we'll declare that the observed data are inconsistent with our simulated model. Here is the [link](#) to the section in the textbook.

Question 1.9: Calculate the proportion of simulated statistics in Question 1.8 greater than or equal to the observed statistic.

Hint: `np.count_nonzero` usage can be found [here](#).

```
[87]: proportion_greater_or_equal = np.count_nonzero(simulated_statistics >=
↳ observed_statistic) / num_repetitions
proportion_greater_or_equal
```

```
[87]: 0.102
```

By convention, we often compare the proportion we just calculated to 0.05. If the proportion of simulated statistics greater than or equal to the observed statistic is sufficiently small (less than or equal to 0.05), then this is evidence against Emily's model. Conceptually, you may think of this as the case where less than 5% of simulated values are as far or farther away from what we had expected. If this is not the case, we don't have any reason to doubt Emily's model.

This should help you make your own conclusions about Emily Rosa's experiment.

Therapeutic touch fell out of use after this experiment, which was eventually accepted into one of the premier medical journals. TT practitioners hit back and accused Emily and her family of tampering with the results, while some claimed that Emily's bad spiritual mood towards therapeutic touch made it difficult to read her HEF. Whatever it may be, Emily's experiment is a classic example about how anyone, with the right resources, can test anything they want!

Question 1.10: Now, take some time to reflect on the questions below and then, discuss with your peers or take a look at the discussions on the Ed post for this lab.

1. Is the data more consistent with Emily's model (practioners were randomly guessing)?
2. What does this mean in terms of Emily's experiment? Do the TT practitioners' answers follow an even chance model or is there something else at play?

Did you talk to your peers or look at the discussion forum? (True/False)

```
[88]: peer_talk = True
      peer_talk
```

[88]: True

1.2 Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a pdf file for you to submit. **Please save before exporting!**

```
[89]: # should change the directory and file name matching to yours
      !jupyter nbconvert './DS/lab13(SSU).ipynb' --to pdf
```

```
[NbConvertApp] Converting notebook ./DS/lab13(SSU).ipynb to pdf
[NbConvertApp] Writing 46264 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 71062 bytes to DS/lab13(SSU).pdf
```